

ECO387C - Fall 2024

Discussion 2

Raymond He & Husang Kim

1 Interpolation

In macroeconomics (and natural sciences), we are often interested in finding entire functional equations, such as value functions and policy functions. However, we often face two challenging issues when trying to find them:

1. Problems are often too complex for a closed form solution to exist. Even in the simplest settings, we need to resort to very careful calibration in order to obtain a solution without needing a computer to help us find a numerical solution.
2. The functions we are interested in finding in many situations are continuous and defined over infinitely many elements. Yet, computers do not have enough memory to store infinitely many elements, much less perform computationally expensive operations on them.

Thus, we need computationally efficient ways to store and represent data (which also always arrive in discrete intervals even if the true underlying data generating process (DGP) is continuous). Scientists often use **interpolation**, which is a method of finding *new* data points based on finding a *finite and discrete* set of known points.

Interpolation requires at least three inputs:

1. A set of inputs (inputs could be multi-dimensional),
2. A corresponding set of outputs, and
3. A family of *basis functions*

The first two are straightforward for today's purposes (and in empirical work, we do not have the freedom to choose which data points we observe anyway).¹ Suppose the true DGP is the sine function but we do not know that.

¹We will revisit this in future discussion sessions.

- Generate x , a vector of 1001 linearly spaced points along $[-5, 5]$ with `linspace`.
- Generate the vector y by applying $\sin(x)$ to each element of the vector above. Plot the results.
- Create an 11-element vector of data points (x_{data}) and corresponding outputs (y_{data}).

Now, we need to specify how the points should be *connected*. Is it with a straight line? Or should it be curved? If so, how much curvature should we impose? In other words, what is the family of *basis functions*? There are many families, but we will focus on the following two today because they have appealing numerical properties:

1. Linear interpolation, and
 2. Cubic interpolation.
- Use the `interp1` command to obtain the corresponding y_{est} for x .
 - Generate $y_{\text{data}} = \sin(x_{\text{data}})$
 - Plot y_{est} and y_{data} over x_{est} in the same plot.
 - Which family of basis functions was closer to the true function?

2 Root-Finding/Optimization

Equation solving is one of the (arguably most) important problems in economics. Consider the demand equation:

$$q(p) = 0.1p^{-0.2} + 0.7p^{-0.5}$$

What is the price that clears the market for a quantity of two units?

Once again, we will often run into problems where closed-form solutions are simply impossible and we need to resort to a numerical solution.²

There are many different algorithms and methods for solving equations but they can broadly be sorted into two categories: **direct** and **iterative** methods. A method is considered direct if it is completed in a predetermined number of steps. Iterative methods take an initial condition and iterates upon it until the algorithm converges to the true solution (or is within some pre-specified tolerance threshold).

²Abel's impossibility theorem states that there are no closed-form solutions to general polynomials of degree $n \geq 5$.

A non-exhaustive list of the most popular algorithms is the bisection method, Newton's method, the secant Method, the golden search method, Brent's method, and Nelder-Meade. There is no 'best' algorithm for every type of problem. Some of the ways they differ from each other are:

1. Convergence guarantee
2. Speed
3. Robustness
4. Reliance on differentiability

In MATLAB, the primary in-built root-finder is `fzero`, and the two main optimization problem solvers are `fmincon` and `fminsearch`.³ The former is suitable when solving a **constrained** optimization problem while the latter solves **unconstrained** problems.

- Solve for the value of x that maximizes $f(x) = -x^2 + 5x - 28$.
- Solve for the value of (x, y) that maximizes the function $f(x, y) = x^2 + y^2$ subject to the constraint $x + y \leq 1$, $x > 0$, and $y > 0$.

In practice, the initial guess you supply can be extremely important. Assuming no human programming error, a poor initial guess is a common reason behind algorithms failing to find convergence, or converge to different solutions if your problem doesn't have a unique solution. On the other hand, if you know that your problem has a unique solution, then it is best practice to experiment with different initial guesses - they should all converge to the same solution!

3 Numerical Integration

In many computational applications, we will need to compute the expected value of functions, which usually means computing definite integrals in the form:

$$\int_I f(x)\omega(x)dx$$

where $\omega(x)$ is a weighting function. As you may expect, computing this definite integral is usually not analytically feasible, hence we need to resort to numerical integration, or *quadrature*. All quadrature methods work by approximating the integral with a weighted sum of function evaluations, or

$$\int_I f(x)\omega(x)dx \approx \sum_{i=0}^N w_i f(x_i)$$

³These commands find the minimum because MATLAB was built with engineers, who encounter minimization problems, in mind. As economists who usually solve maximization problems, you will need to transform your problem accordingly.

We need to make 3 important choices:

1. The x_i 's, or **quadrature nodes**,
2. the w_i 's, or **quadrature weights**, and
3. N , the number of function evaluations.

There are many methods (Simpson's rule, Midpoint rule, the Trapezoid rule), but for today, we will only cover **Gaussian quadrature**.⁴ This method picks the x_i 's and w_i 's to satisfy $2N$ 'moment-matching' conditions (for a given N):

$$\int_I x^k \omega(x) dx = \sum_{i=1}^N w_i x_i^k \text{ for } k = 0, \dots, 2N - 1.$$

When $\omega(x) \equiv 1$, we are calculating the area under a curve, and this is called Gauss-Legendre quadrature.

- Using the function integral, compute

$$\int_1^4 x^3 - 2x^2 + 3x dx$$

This is extremely useful for us because when $\omega(x)$ is the **probability density function**, of the random variable $\tilde{X}$, Gaussian quadrature discretizes $\tilde{X}$ and x_i becomes individual mass-points with probabilities w_i . I.e. together, they mimic the distribution $\tilde{X}$ is drawn from by having the same moments less than $2N$:

$$\sum_{i=1}^N w_i x_i^k = \mathbb{E}(\tilde{X}^k) \text{ for } k = 0, \dots, 2N - 1.$$

Once the nodes and weights have been chosen, computing the final integral is reduced to simply computing the following weighted sum:

$$\mathbb{E}(f(\tilde{X})) \approx \sum_{i=1}^N w_i f(x_i)$$

- Suppose the random variable $\tilde{X}$ is drawn from the normal distribution with mean 3 and variance 4.
- Use the `qnwnorm` function to calculate the nodes and weights for $N = 3$.
- Plot the x_i 's against the w_i 's. How closely does it resemble the true distribution?
- Increase N to 30. Plot the results and compare again.

⁴There are also Monte Carlo methods, but they are extremely inefficient relative to Gaussian quadrature and should only be used when the number of dimensions in your integral is 4 or more.